```
print(counts)
plot_histogram(counts)
plt.show()
{'111': 1024}
```

I'll explain this step-by-step in **very simple terms**, then connect each step to the **theory you need for viva**.

🧠 PART 1: Basic Idea (Before Code)

You are working with **quantum bits (qubits)**:

- Classical bit → 0 or 1
- Quantum bit → can be **0, 1, or both at same time (superposition)**

A 2-qubit system has 4 possible states:

[redacted]

⚙️ PART 2: Code Explanation (Line by Line)

Step 1: Imports

Python

```python
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
import matplotlib.pyplot as plt
```

- `QuantumCircuit` → used to build circuit
- `Statevector` → shows exact quantum state (before measurement)
- `AerSimulator` → simulates quantum computer
- `plot_histogram` → shows output graph

Step 2: Create Circuit

Python

```
qc = QuantumCircuit 2  2
```

- 2 qubits (q0, q1)
- 2 classical bits (for storing result)

Initial state:

Step 3: Hadamard Gate (Superposition)

```python
qc.  0
```

👉 Applied on qubit 0

What happens?

It converts:

So system becomes:

Step 4: X Gate (NOT Gate)

```python
qc.  1
```

👉 Applied on qubit 1

Flips:

Now state becomes:

Step 5: CNOT Gate (Entanglement)

```python
qc.   0  1
```

👉 Control = qubit 0
👉 Target = qubit 1

Rule:

- If control = 1 → flip target
- If control = 0 → do nothing

Apply to each part:

State	Result
	01>
	11>

Final state:

[redacted]

Step 6: Measurement

```python
qc.         0 1    0 1
```

Now quantum state collapses to classical result.

📊 PART 3: Output Understanding

Statevector Output

[redacted]

Mapping:

[redacted]

[redacted]

👉 Only these states exist:

- |01>
- |10>

Histogram Output

[redacted]

👉 About 50% each:

- |01>
- |10>

❗ Important:
You **never get 00 or 11**

🔗 KEY THEORY (VERY IMPORTANT FOR VIVA)

1. Superposition

A qubit can exist in multiple states at once.

Mathematically:

[redacted]

Where:

[redacted]

---

2. Entanglement (CRITICAL CONCEPT)

Your final state:

[redacted]

This is an **entangled state**.

👉 Meaning:

- If you measure one qubit → you instantly know the other
- They are **correlated**

Example:

- If first qubit = 0 → second must be 1
- If first qubit = 1 → second must be 0

---

3. CNOT Gate Logic

Control	Target	Output
0	0	0
0	1	1
1	0	1
1	1	0

---

4. This State Name

Your final state is:

[redacted]

This is a **Bell State (Ψ^+)**

🧠 PART 4: Toffoli Gate (3-Qubit Logic)

Code:

```python
qc = QuantumCircuit 3  3

qc.  0
qc.  1
qc.    0  1  2
```

Step-by-step

Initial:

[redacted]

After X gates:

[redacted]

Toffoli Gate (CCX)

👉 2 controls (q0, q1)
👉 1 target (q2)

Rule:

- Flip target ONLY if both controls = 1

So:

[redacted]

Output:

[redacted]

Always same → deterministic

🔗 Toffoli Gate Theory

1. Also called:

- Controlled-Controlled-X (CCX)
- Universal reversible gate

2. Logic Table

q0	q1	q2	Output q2
0	0	0	0
0	1	0	0

q0	q1	q2	Output q2
1	0	0	0
1	1	0	1

---

3. Importance

- Used in:
  - Quantum algorithms
  - Error correction
  - Reversible computing

⚠️ COMMON MISTAKE (IMPORTANT)

Many students think:

> "H + CNOT always gives entanglement"

❌ Wrong

👉 You must check initial states carefully
👉 Here, X gate changed the outcome

🧪 DEBUGGING THINKING (VERY IMPORTANT FOR YOU)

If output is wrong, check:

1. Initial state
2. Gate order (VERY sensitive)
3. Control vs target qubit
4. Measurement mapping

📌 FINAL SUMMARY

- H → creates superposition
- X → flips qubit
- CNOT → creates entanglement
- Result → Bell state (Ψ^+)
- Toffoli → 2-control logic gate

If you want next level:

- ↳ I can derive this **mathematically using matrices**
- Or ↳ show how this connects to **quantum teleportation (advanced viva topic)**